XploitSQL: Advancing Adversarial SQL Injection Attack Generation with Language Models and Reinforcement Learning

Daniel Leung
Simon Fraser University
School of Computing Science
Burnaby, British Columbia, Canada
thl28@sfu.ca

Omar Tsai
Simon Fraser University
School of Computing Science
Burnaby, British Columbia, Canada
omar.tsai@sfu.ca

Kourosh Hashemi
Simon Fraser University
School of Computing Science
Burnaby, British Columbia, Canada
kourosh_hashemi@sfu.ca

Bardia Tayebi
University of Isfahan
School of Computing Science
Isfahan, Iran
bardya.tayebi@mehr.ui.ac.ir

Mohammad A. Tayebi
Simon Fraser University
School of Computing Science
Burnaby, British Columbia, Canada
tayebi@sfu.ca

Abstract

SQL injection (SQLi) compromises database-driven applications by enabling attackers to insert malicious SQL commands via input fields, potentially leading to unauthorized access, data manipulation, or system compromise. In recent years, alongside the development of various rule-based Web Application Firewalls (WAFs) aimed at mitigating SQL injection attacks, there has also been a notable rise in the utilization of machine learning and deep learning techniques to address this issue. Although significant progress has been made in these studies, detecting and mitigating SQLi-related attacks continues to present a significant challenge. A crucial factor contributing to the lack of extensive SQLi detection solutions is the absence of a comprehensive testing methodology. In this work, we introduce XploitSQL—an innovative approach to advance adversarial SQL injection generation by leveraging language models and reinforcement learning. Our model is trained to produce evasive SQLi samples, enhancing the robustness of SQLi detection models and offering opportunities for more comprehensive detection strategies. To assess the efficacy of the proposed method, we employed state-of-the-art SQL injection detection models in conjunction with commercially available web-based firewalls. Across all tested detection models, detection rates declined when faced with evasive samples generated by XploitSQL. Furthermore, our model outperforms existing methods for generating attack samples.

CCS Concepts

• **Security and privacy** → **Database and storage security**; *Database activity monitoring*; • **Computing methodologies** → Natural language processing.

Keywords

Database security, SQL injection, Language models, Reinforcement learning

ACM Reference Format:
Daniel Leung, Omar Tsai, Kourosh Hashemi, Bardia Tayebi, and Mohammad A. Tayebi. 2024. XploitSQL: Advancing Adversarial SQL Injection Attack Generation with Language Models and Reinforcement Learning. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24), October 21–25, 2024, Boise, ID, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3627673.3680102

CIKM '24, October 21–25, 2024, Boise, ID, USA

ACM ISBN 979-8-4007-0436-9/24/10
https://doi.org/10.1145/3627673.3680102

1 Introduction

SQL injection (SQLi) attacks are a significant security threat to web applications that use SQL for database management. By exploiting input validation flaws, attackers can inject malicious SQL commands, potentially leading to data breaches or system control. SQLi is one of the oldest vulnerabilities and has remained in the OWASP Top 10 for over twenty years [2]. In 2023 alone, 2,159 SQL injection vulnerabilities were reported and assigned CVE identifiers [3].

Preventing SQLi attacks is vital for safeguarding web and data security. SQLi detection strategies generally fall into two categories: rule-based [15, 29] and machine learning (ML) approaches [14, 18, 19]. Traditionally, Web Application Firewalls (WAFs) [6, 20] used in detecting SQLi attacks heavily rely on predefined detection rules employing semantic analysis. However, the adaptable nature of SQL syntax presents challenges in covering all potential SQLi scenarios, leading to diminished detection efficacy [28]. To address this, researchers increasingly explore machine learning techniques [14, 17, 18] for SQLi detection. These methods typically entail converting queries into word vectors and extracting consecutive characters as features to identify malicious queries. Despite notable progress, these approaches are still susceptible to attacks, posing a significant challenge that warrants further attention.

One of the primary causes of failure in SQLi detection models is the absence of thorough testing before deploying such solutions [7]. Implementing a rigorous testing approach enables developers of detection models to gain deeper insights into the weaknesses of their solutions and effectively address them.

The purpose of red teaming [10] in cybersecurity is to proactively simulate real-world attacks, aiming to pinpoint vulnerabilities, evaluate defense solutions, and enhance an organization's overall security posture. Inspired by red teaming methodologies, we introduce XploitSQL a novel and efficient approach to generate SQL injection attacks. XploitSQL utilizes language models and reinforcement learning to make this process more effective. Through iterative refinement and interaction with the target detection model, XploitSQL demonstrates an enhanced capability to generate increasingly evasive SQLi samples, effectively evading detection mechanisms.

Our proposed model begins with training a language model specialized in generating SQL queries, establishing the foundation. Subsequently, reinforcement learning is introduced through an actor-critic setup, where the pre-trained language model serves as both actor and critic. Following the completion of reinforcement learning training, we preserve the actor model and discard the critic model. A carefully crafted reward function motivates the actor model to produce SQL queries that meet syntactic correctness and perform optimally in predefined evaluation metrics for attacking a target SQLi detection model.

Extensively evaluated across diverse detection systems and in comparison with the baseline models, XploitSQL demonstrates remarkable efficacy in most of the test scenarios. The experimental results not only highlight a significant decrease in the performance of both WAF and ML-based SQLi detection models but also show that our model outperforms the baseline models. The SQL injection (SQLi) detection rate for the evaluated methods significantly decreases by up to 47% when using adversarial samples generated by XploitSQL. Our model either surpasses or achieves the same performance as baseline models in five out of six detection models which shows the robustness XploitSQL in generating adversarial SQLi samples.

In summary, our contributions are as follows:

- We propose a methodology leveraging the capabilities of LLMs (Large Language Models) and RL (Reinforcement Learning) to generate targeted evasive SQL injections.
- To ensure the effectiveness of the generated attack samples, we test them against state-of-the-art machine learning-driven models and WAFs developed by major vendors.
- To ensure the comprehensiveness of our methodology, we compared the model with baselines to assess its performance.

In this paper, Section 2 provides an overview of related work. Section 3 introduces the background and problem statement. Section 4 details our proposed approach for generating adversarial SQL injection samples. Section 5 analyzes and discusses the experimental results. Section 6 explores further applications of the proposed model, while Section 7 offers concluding remarks for the paper.

2 Background and Related Work

2.1 Exploiting SQL Injection

SQL injection is a type of code-injection attack that exploits vulnerabilities by manipulating user input, causing the system to interpret it as SQL code. This manipulation can result in unauthorized access to databases. SQL injection can be categorized into three distinct types. Firstly, there's in-band SQL Injection, also known as classic SQL injection, where the attacker uses the same communication channel for both injecting SQL code and retrieving data. Secondly, out-of-band SQL Injection involves using a different channel to extract data from the database, separate from the injection channel. This could include techniques like DNS or HTTP-based data retrieval. Lastly, inferential SQL Injection doesn't directly reveal the result of the SQL injection in the application's response. Instead, the attacker infers the success or failure of their SQL query based on differences in the application's behavior.

The primary responsibility for ensuring input data validation and code vulnerability analysis falls on application developers. However, considering real-world experiences, even following secure programming best practices, vulnerabilities can still emerge. This emphasizes the importance of an additional layer of protection. WAFs serve as critical defense mechanisms by conducting thorough packet inspections on network traffic between clients and servers. By analyzing the exchanged data, WAFs can detect potential attacks, including SQLi, that may have slipped through during implementation. The subsequent section will delve into this topic.

2.2 Web Application Firewall

Web Application Firewalls (WAFs) are essential defenders of enterprise web systems, shielding them from malicious attacks. Operating as a protective barrier around the web applications they safeguard, WAFs analyze incoming HTTP messages to discern whether they should be permitted to reach the intended destination or blocked. This decision-making process relies on a set of rules designed to identify and intercept potential attack patterns, thus ensuring the robust security and integrity of the entire web infrastructure. WAFs come in various forms: signature-based, ML-driven, or hybrid, which leverage the strengths of both approaches.

Signature-based WAFs protect applications only based on predefined attack signatures. These signatures are patterns or rules derived from known threats. However, signature-based WAFs can become increasingly complex over time, necessitating advanced expertise to update as new attacks emerge. Their reliance on human intervention introduces potential vulnerabilities, such as implementation bugs and configuration errors. Without careful management, they are prone to high rates of false positives and false negatives.

Contrary to signature-based WAFs, ML-driven WAFs capitalize on extensive datasets of both legitimate and malicious traffic to extract meaningful patterns. This iterative process enables them to continually enhance their understanding of evolving attack vectors.

Appelt *et al.* [5] propose a hybrid method merging machine learning and evolutionary algorithms to autonomously evaluate WAFs' attack detection capabilities. This method generates a diverse array of attacks, assesses their identification by the WAF, and adapts based on results. By learning from both blocked and bypassed tests, this method prioritizes tests with patterns likely to bypass the firewall. Ito *et al.* [13] employ a character-level Convolutional Neural Network with extensive global max-pooling to extract features from HTTP requests and classify them as normal or malicious. Betarte *et al.*'s [8] approach addresses various scenarios based on data availability and offers two solutions: a multi-class approach for scenarios with both legitimate and malicious data, and a one-class solution for scenarios with only legitimate data.

2.3 Evading SQLi Detection Models

The focus of this research domain is primarily on developing defense solutions rather than attack methods, driven by the pressing need to protect systems from evolving threats. However, significant work is also dedicated to demonstrating the weaknesses of detection systems designed for various vulnerabilities, leading to the development of more ML-driven open-source solutions. Xu *et al.* introduced a technique using a genetic algorithm to generate adversarial examples that can evade PDF malware classifiers [27]. Anderson *et al.* proposed a model to bypass various malware detection systems by modifying malware samples through semantically invariant transformations, relying solely on the score provided by the victim classifier [4]. When attackers lack direct access to a specific system but have some knowledge of it, the authors suggest training a substitute model to simulate the target system. Several studies fall into this category, crafting attacks across different domains [9, 12, 16, 24].

Regarding SQLi, malicious actors continuously evolve their techniques to create more sophisticated and evasive samples, exploiting weaknesses in ML-driven or signature-based WAFs. They use methods such as obfuscation and mutation to alter the structure and appearance of attacks, bypassing signature detection. Additionally, they employ adversarial machine learning to craft inputs specifically designed to evade ML models, exploit model overfitting, and iteratively refine their techniques based on WAF responses. By mimicking legitimate queries and using a variety of encoding methods, attackers ensure their malicious activities remain undetected, constantly staying ahead of security measures.

Our proposed solution, XploitSQL aligns with two recently proposed models designed to evade SQLi detection systems, WAF-A-MoLE [25] and AdvSQLi [21]. WAF-A-MoLE uses guided mutation fuzzing to lower the confidence scores of the classifier. In each mutation cycle, the mutant with the lowest confidence score is selected as the basis for the next cycle. However, this approach is less effective against detection since the final result follows a straightforward path through a mutation tree. AdvSQLi [21] is a weighted mutation strategy based on context-free grammar to generate an extensive array of equivalent SQLi payloads. It leverages the Monte-Carlo tree search to efficiently select the adversarial payload from this extensive space. XploitSQL, as elaborated in Section 4, tackles the same problem addressed by WAF-A-MoLE and AdvSQLi, but it takes a different approach. Our experimental evaluations show that XploitSQL outperforms the other two comparable models across the majority of cases.

3 Problem Statement

We approach the SQLi attack generation problem through an adversarial lens. In the realm of cybersecurity, it's essential to perceive challenges from this standpoint and enhance defensive strategies based on weaknesses discovered in this process. This process, often termed red teaming versus blue teaming, focuses on defending against the red team's attacks.

Given an SQLi detection system, the primary objective of an adversary is to develop a model capable of generating SQL injection queries that bypass the detection system. The detection system classifies queries into two categories: benign ($y = 0$) and malicious ($y = 1$). We assume a black-box attack scenario, where the model has no access to the detection system's internal workings but can observe its output indicating whether a query is flagged as benign or malicious. The goal is to produce SQL injection queries that are malicious but are incorrectly classified as benign by the SQLi detection system. As the starting point of this problem space, a set of SQL injection queries that the detection system identifies as malicious is given. The challenge for an adversary is that, for a given test query x, to produce a modified query x_{gen} that evades detection and is classified as benign by the detection system.

In a black-box setup, the attacker's efforts are limited to the testing phase without access to the internal architecture of the target SQLi detector. Essentially, the attacker can only input data and observe the resulting output, without any further access or insight. Despite these limitations, attackers can still manipulate database requests, potentially modifying or accessing sensitive data or causing irreversible damage to databases. Moreover, executing SQLi attacks does not require advanced hacking skills, as many free or commercially available tools are readily available online. With some practice, attackers can also develop their own scripts to exploit SQLi vulnerabilities effectively.

4 Methodology

Our proposed approach for SQLi attack sample generation, XploitSQL, utilizes a hybrid technique by combining large language models with reinforcement learning, as illustrated in Figure 4. Initially, a language model is trained to generate SQL queries, serving as the foundational model. Following the training of an initial language model, reinforcement learning is introduced into the architecture—following an actor-critic method. Here, the pre-trained language model acts as both the actor and the critic. Once training is completed using reinforcement learning, the actor model is then preserved while the critic model is discarded. The reward function is carefully designed to incentivize the actor model towards generating SQL queries that not only adhere to syntactic correctness but also show optimal performance according to the specified evaluation metrics.

4.1 Pre-trained Language Model

Modern LLMs, predominantly based on the transformer architecture [26] features a self-attention block, a pivotal innovation distinguishing them from prior NLP models. This block computes a weighted average of the input sequence, enabling the model to focus on specific elements rather than merely the next character. By leveraging this capability, transformers can discern crucial parts of the input sequence, resulting in improved performance across various NLP tasks.

LLMs, when fine-tuned for specific tasks, demonstrate exceptional performance. This not only significantly reduces the time and effort required to train and develop language models from scratch but also enhances their effectiveness for targeted tasks. One prominent example of such an LLM is the Text-to-Text Transfer Transformer (T5) model [23]. This model operates on a similar principle to the original transformer model, featuring an encoder-decoder architecture pre-trained through unsupervised learning. It operates by providing data in a text-to-text format to the model,

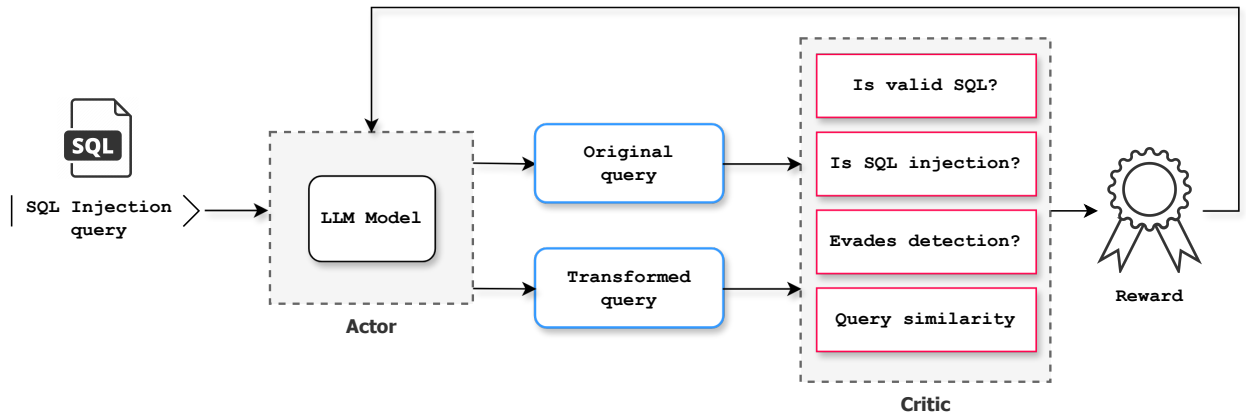

Figure 1: Workflow of LLM and RL models in training XploitSQL

prompting it to generate output text corresponding to the input text. Additionally, since the dataset comprises web-extracted text, this enables the model to learn across a broad spectrum of tasks.

As the starting point for creating a refined SQL injection sample, XploitSQL uses a T5 model that is pre-trained for generating SQL queries. The rationale behind this choice lies in the fact that this model is already trained for generating SQL queries. By fine-tuning it further, we can steer its focus towards generating SQL injection queries, which represent a more specialized type of SQL query. This approach not only conserves computational resources compared to training a base model from scratch but also promises performance enhancements for our final model.

For a language model, *prompt tuning* is a pivotal factor influencing model performance. Within our training methodology, we thoroughly explore a multitude of potential keywords and sentences capable of enhancing the model's proficiency in generating high-quality samples. Subsequently, the model is subjected to a spectrum of novel prompts derived from these elements. The queries generated in response undergo a thorough evaluation, allowing us to discern which keywords and phrases yield superior outcomes. This iterative refinement process contributes significantly to optimizing model performance.

4.2 Fine-tuning with RL

Using RL for LLM-based generation poses practical challenges, notably training instability arising from the vast combination of possible actions. This issue is addressed in [22], which introduces a performant RL algorithm named NLPO. XploitSQL also employs NLPO to improve SQL injection queries generated by the T5 model. Through top-p sampling, NLPO learns to mask out less relevant tokens during training, thereby reducing the action space and enhancing stability. By periodically updating masking on the current policy, NLPO balances task relevance and training efficiency.

To fine-tune the language model with RL, XploitSQL focuses on recognizing relevant tokens and filtering out non-relevant tokens. In this process, keywords and commonly used identifiers are kept while less essential ones are dropped. This effectively reduces our action space—making the language model prioritize more important tokens. For example, SQL tokens such as `SELECT`, `FROM`, `WHERE` and `JOIN` are prioritized, while rare or special tokens are masked out as they are less likely to appear in typical queries. The emphasis on more relevant SQL tokens leads us to a model that generates more accurate and contextually appropriate queries.

To enhance the employed LLM for generating increasingly evasive SQLi samples, we integrate reinforcement learning techniques. Our primary objective is to evade the detection model, and for this purpose, we utilize an actor-critic framework. In this framework, both the actor and the critic are based on the same LLM, albeit as two distinct copies of the base LLM model. The actor is tasked with learning the structure of SQLi and generating queries that evade the target SQLi detection model. Meanwhile, the critic evaluates the generated queries using the reward function defined in Section 4.3. This evaluation includes assessing whether the queries bypass the target SQLi detection model, thereby facilitating a reinforcement learning feedback loop that promotes the generation of appropriate evasive SQLi samples.

This fine-tuning process is iterative, wherein the model is provided with queries generated in its previous iteration. This setup enables the model to learn from its mistakes and improve over time. Initially, in the first iteration, only the values from the dataset are utilized, as there are no previous model-generated outputs available. However, starting from the second iteration onwards, the model begins to leverage the generated outputs from the previous iteration for training. This is achieved by having the model from the last iteration produce a new set of queries based on the training dataset. Subsequently, this set of new queries is evaluated and utilized as the training dataset for the next iteration. Additionally, to provide the model with references to the original dataset, not just the generated queries from the last iteration, 10% of the original set is also included in the input of each round of iterations.

4.3 Reward Model Training

To assess if the generated query effectively evades the detection model, we incorporate a reward model to evaluate its performance.

This model assigns a score to the query, which then serves as a reward for refining LLM through the RL learning process. The following are four metrics we use in this process.

- ◇ *Validity measure.* This metric evaluates whether the generated query is a valid SQL query or not. It's distinct from checking whether the query is an SQL injection attempt because it's plausible for the generated query to be synthetically invalid but bypass a SQLi detection model.
- ◇ *Injection measure.* This metric evaluates whether the generated query is an SQLi attempt. Employing this approach enables tiered rewards based on multiple metrics, rather than a binary reward system, facilitating more effective learning. Note that not all SQLi queries can bypass the detection model.
- ◇ *Evasion measure.* This metric ensures evasion capability, employed both independently and in conjunction with the two preceding metrics. Their collective assessment forms the base-tier reward, further augmented by the final metric.
- ◇ *Similarity measure.* This measures query likeness between the input and generated queries. Unlike prior metrics, it's not binary but falls within a 0-1 range. A score of 0 signifies no similarity, while 1 denotes exact replication, indicating identical queries.

The reward function assigns scores to different metrics for each query, which contribute to the overall reward. It has two parts: the *base reward* and the *similarity multiplier*. The base reward is determined by comparing the metrics of the generated query to the original query considering validity, injection and evasion measures. Improvements in these metrics earn a reward, while regressions incur a penalty. Finally, the base reward is adjusted by the similarity measure, which penalizes very high or low similarity but is more lenient for evasive queries. The similarity measure helps ensure the model generates meaningful modifications. Very low similarity suggests random outputs, while very high similarity means the model is copying the query. Both extremes are penalized. However, if the query can already evade detection, the penalty for high similarity is reduced to encourage similar evasive queries. This tiered reward model guides XploitSQL toward creating more evasive queries.

5 Experimental Evaluation

This section presents our experimental design and results, beginning with a description of the data characteristics.

5.1 Experimental Design

In this section, we detail our experimental design aimed at quantitatively assessing the efficacy of our proposed solution in generating adversarial samples.

Dataset. Our reference dataset used for generating adversarial samples, referred to as the *original dataset*, is a combination of multiple datasets. It consists of a total of 35,574 samples, comprising 15,134 malicious samples and 20,440 benign samples. These initial samples are used as the basis to generate new samples. The dataset resulting from the iterative training process generated by our proposed model is denoted as the *adversarial dataset*.

Detection Models. To showcase the effectiveness of XploitSQL compared to baseline approaches, we select two sets of SQLi detection models, ML-based models and WAFs. ML-based models are widely used for text classification tasks such as spam detection and sentiment analysis. These models preprocess text data and convert it into numerical features using techniques like TF-IDF or word embeddings. Similar solutions can be used for SQLi detection [11, 18].

In our experimental results, we employ two machine learning models, CNN and LSTM, as SQLi detection models. To evaluate the real-world efficacy of our proposed model, we use four WAFs from major vendors, including Amazon Web Services (AWS), F5 and the cutting-edge open-source WAF, ModSecurity. All of these SQLi detection models have demonstrated strong performance in classifying malicious and benign SQLi samples, and we aim to assess their resilience in detecting adversarial samples generated by our proposed model and tested baseline models.

Baseline Models. Baseline methods include approaches in the literature aimed at generating evasive SQLi samples, against which we compare our proposed method, as introduced below:

- **WAF-A-MoLE.** This model leverages query mutation to generate evasive SQLi samples [25].
- **AdvSQLi.** This model utilizes Monte-Carlo tree search to effectively select adversarial SQLi samples [21].
- **LLM-SQLi.** This model is a variant of XploitSQL that does not leverage fine-tuning with RL. In other words, it comprises solely an LLM component.

Evaluation Measures. To assess the impact of our adversarial generation algorithm on SQL injection classification methods, we define a metric called *Penetration Rate* (P) as follows: $P = \frac{FN}{TP+FN}$. FN represents the count of SQLi queries incorrectly classified as benign, while TP denotes the number of SQLi payloads correctly classified as attacks. It's important to note that the penetration rate signifies the proportion of SQLi attacks that were either missed or incorrectly classified as benign.

5.2 Results

The bypass rates for six detection models—CNN, LSTM, AWS, F5, Fortinet, and ModSecurity—using the original dataset are 9.85%, 6.35%, 9.78%, 40.23%, 100%, and 10.54%, respectively. It's noteworthy that applying XploitSQL and baselines is expected to increase the bypass rate of detection models, thereby highlighting their vulnerability to SQLi. Conversely, a higher increase in the bypass rate resulting from an attack model demonstrates the effectiveness of that model.

Table 1 presents the bypass rate of XploitSQL compared to baseline models. For both ML-driven models, CNN and LSTM, XploitSQL outperforms the baselines. Notably, XploitSQL surpasses AdvSQLi, the second-best attack model, by 8% and 6% for the CNN and LSTM models, respectively. For the evaluated WAFs, AWS, F5, Fortinet, and ModSecurity, XploitSQL significantly outperforms the baselines for two of them, while performing weaker for one. Specifically, XploitSQL exceeds the second-best model by 4% for AWS and by 29% for ModSecurity but underperforms by 23% for F5. For Fortinet, all models achieve the same level of performance.

XploitSQL's main components, LLM, and RL, collaboratively iterate to produce increasingly evasive queries. The results of LLM-xSQLi demonstrate the performance of XploitSQL without RL-based iterative training. These results affirm the significant contribution of this component to XploitSQL's performance.

Injection Source	CNN	LSTM	*AWS*	*F5*	*Fortinet*	*ModSecurity*
WAF-A-Mole	22.90%	10.22%	36.48%	48.32%	100%	23.31%
AdvSQLi	36.27%	26.43%	41.03%	**78.49%**	100%	8.89%
LLM-SQLi	38.90%	29.02%	52.96%	19.80%	100%	40.61%
XploitSQL	**44.38%**	**32.60%**	**57.04%**	55.27%	100%	**52.74%**

Table 1: The Bypass rate of XploitSQL and baselines for the evaluated SQLi detection models

In comparing the baselines, AdvSQLi outperforms WAF-a-MoLE for all evaluated models except one. While both algorithms utilize query mutation, AdvSQLi employs a more focused approach, enabling precise alterations within the query segment while preserving the overall SQL command structure. This preserves the payload's functionality and malicious intent. Moreover, AdvSQLi utilizes Monte Carlo tree search to intelligently explore and select the most effective mutations based on context-free grammar rules.

An interesting observation from our experimental results is the superiority of ML-driven solutions over traditional WAFs. This advantage stems from the adaptive learning capabilities of ML-driven WAFs. In contrast, traditional WAFs typically rely on static rule-based approaches, which lack the ability to adapt dynamically. Furthermore, this finding challenges recent claims made by major vendors about using ML-driven solutions in their security products, including WAFs. An unexpected result is the 100% bypass rate for Fortinet WAF, signaling a significant vulnerability in this specific configuration. This finding suggests a potential issue with the Fortinet WAF version used during testing, which might not be effectively interpreting or filtering URL parameters, allowing SQL injection queries to pass through undetected. This vulnerability indicates that the WAF model may not be correctly analyzing the structure or content of the web requests it processes.

Figure 2 depicts how XploitSQL's performance evolves in each iteration across the evaluated WAFs. We observe a consistent upward trend in all cases, with minor fluctuations in some instances, ultimately converging to an overall increasing trend. This shows the contribution of XploitSQL's iterative learning to its performance.

A unique aspect of XploitSQL is its adaptability in identifying vulnerabilities in the target detection model. This is achieved through its iterative interactions with the target model, where it learns the model's weaknesses by sending various types of SQLi queries and receiving feedback through the reward function defined in the RL component. In iteration $N + 1$, the query generated in iteration N along with a fine-tuned prompt is passed to the LLM. This iterative process continues until a sufficiently evasive query is generated. Figure 3 illustrates the process of inference, showing a sample original query and the final evasive query generated after the 16th round of interaction between XploitSQL and the target model. For example, in earlier iterations, such as iterations 4 and 8, the LLM only learns to add random strings and unrelated tokens, resulting in queries that fail to evade detection. By iteration 16, the LLM refines the query, producing a more evasive query capable of bypassing the detection model.

In the experimental evaluation, we aimed to achieve two primary goals: generating SQLi samples with higher evasiveness than those in the training dataset and doing so more effectively than existing models. For the first goal, the results clearly demonstrate the effectiveness of our model, as evidenced by the performance drop in six different detection models. For the second goal, our model outperforms or matches the baseline in 11 out of 12 comparisons, showcasing the superiority of XploitSQL.

6 Discussion

6.1 Generating Insights for Defenders

Targeting security products and analyzing them from an adversarial perspective is a widespread practice in the software industry. An essential application of XploitSQL is enhancing SQL injection detection tools, such as the WAFs evaluated in this study. WAF developers can leverage XploitSQL to identify and address vulnerabilities in their products effectively. This process can occur throughout the software development lifecycle, including code reviews, continuous integration and even threat modeling.

On inspecting the bypassed queries by XploitSQL for a rule-based WAF such as ModSecurity, we found a specific pattern that consistently bypassed the WAF; more specifically, the use of the `MIN()` function. One such example is the query `SELECT MIN(injection) FROM database`. The core rule set for ModSecurity is open source, allowing us to explore why the query pattern was bypassed [1]. In our research, we found ModSecurity does not implement any rules for this function (and other functions such as `MAX()`). Furthermore, we can see that with XploitSQL's output, 846 of the 1371 queries generated in the final iteration contained the use of the `MIN()` function. This validates our model has indeed identified a small gap that exists in a rule-based WAF to exploit it.

Further analyzing the `MIN()` function vulnerability, we notice the same pattern for other WAFs as well. For F5, 79% of queries

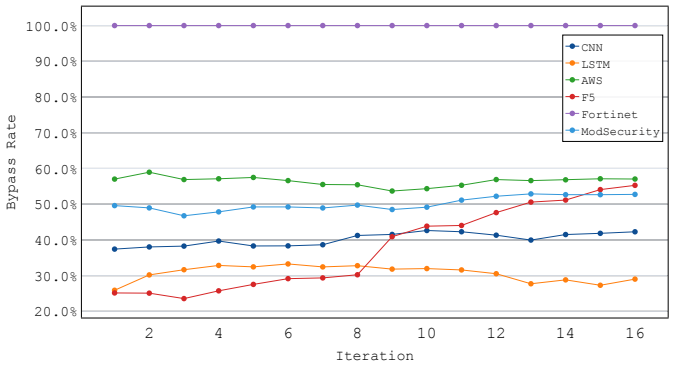

Figure 2: The bypass rate of XploitSQL across various training iterations for evaluated detection models

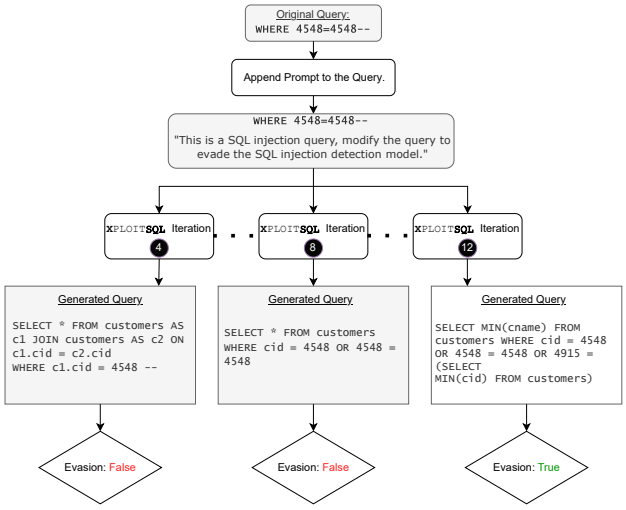

Figure 3: Queries generation across XploitSQL iterations

generated by XploitSQL were `MIN()` queries that bypassed the WAF, whereas for AWS, it was around 50%. This again highlights the practical aspects of reinforcement learning in training XploitSQL for more evasive SQLi generation.

In following responsible disclosure rules, before publishing and with ample time, we will dispatch a comprehensive technical description of our vulnerability analysis to the relevant vendor or the maintainers of open-source tools we tested in this study.

6.2 Exploiting the Cyber Kill Chain

The *Cyber Kill Chain* is a model that outlines the various stages of a cyber attack [28]. XploitSQL can be applied to various steps of the cyber kill chain, including reconnaissance, weaponization, exploitation, and exfiltration. In the reconnaissance phase, attackers typically gather as much information as possible about the target system. XploitSQL offers a new vector for extracting additional system information through SQLi. By training on the target system, the LLM help can classify queries to assist attackers in understanding which types of queries the system is vulnerable to, providing a detailed map of potential weaknesses.

In the weaponization phase, the trained LLM can create a variety of payloads based on the patterns it has learned. This allows attackers to generate specially crafted SQL injection queries tailored to the target system. During the exploitation phase, attackers can use the knowledge gathered from reconnaissance, combined with the LLM's capabilities as an injection generator, to exploit target system vulnerabilities, including known CVE attacks and timing attacks. Finally, in the exfiltration phase, attackers can generate and send exfiltration queries using XploitSQL to extract sensitive information from the target system. As we have outlined, our comprehensive methodology enhances the effectiveness of each phase of the cyber kill chain; making it a powerful tool for testing target systems from an attacker's perspective.

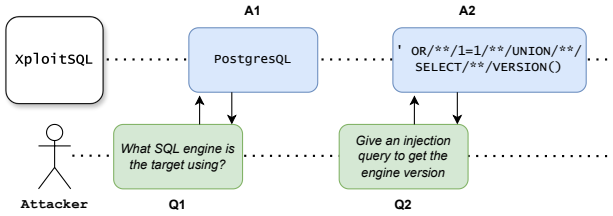

Figure 4: Leveraging XploitSQL for reconnaissance

6.3 Model Limitation and Future Work

The performance of XploitSQL, is directly influenced by the size of the LLM model used. Larger models show superior performance due to their enhanced contextual understanding and advanced learning capabilities. For example, the model used in our study has only 77 million parameters, whereas newer models have billions of parameters, showcasing the significant differences in capability and performance. On the other hand, for the XploitSQL, training process, we required two copies of the same LLM to implement the actor-critic reinforcement learning model. This introduced additional inefficiencies in computational resource usage, ultimately impacting the efficiency of our final model.

Building on the methodologies outlined in this paper, we plan to conduct experiments using larger models to evaluate the correlation between model size and performance. Refactoring the framework may also become necessary to enhance training efficiency, thus mitigating the substantial computational resource demands. We believe leveraging LLM capabilities for security applications can open up promising research avenues, and our work stands as one of the pioneering efforts in this domain, warranting further investment and exploration.

7 Conclusion

There has been substantial investment in developing tools to address web application vulnerabilities, such as SQL injection. Recently, machine learning applications in cybersecurity have shown significant potential in enhancing these tools. However, these methods are still not performing as expected, and SQL injection remains a commonly exploited vulnerability. Additionally, adversarial agents are continually prepared to exploit security weaknesses. Therefore, improving detection tools requires a thorough analysis of current vulnerability detection methods. In this work, we present an innovative method for generating adversarial SQL injection attacks by leveraging the integration of large language models and reinforcement learning. Our model is designed to produce evasive SQLi samples, thereby enhancing the effectiveness of SQL injection detection systems and enabling more comprehensive detection strategies. Extensive evaluations reveal the superior efficiency of our method compared to baseline approaches. In future work, we plan to enhance our approach by capitalizing on the capabilities of large language models to improve their generalization, ensuring that our adversarial samples can effectively evade detection across a diverse range of machine learning models.

References

[1] Owasp modsecurity core rule set. https://owasp.org/www-project-modsecurity-core-rule-set/. Accessed May 18, 2024.

[2] Owasp top ten. https://owasp.org/www-project-top-ten/. Accessed May 14, 2024.

[3] Vulnerabilities by types. https://www.cvedetails.com/vulnerabilities-by-types.php. Accessed May 14, 2024.

[4] Anderson, H. S., Kharkar, A., Filar, B., and Roth, P. Evading machine learning malware detection. *black Hat 2017* (2017).

[5] Appelt, D., Nguyen, C. D., Panichella, A., and Briand, L. C. A machine-learning-driven evolutionary approach for testing web application firewalls. *IEEE Transactions on Reliability 67*, 3 (2018), 733–757.

[6] Applebaum, S., Gaber, T., and Ahmed, A. Signature-based and machine-learning-based web application firewalls: A short survey. *Procedia Computer Science 189* (2021), 359–367.

[7] Aydos, M., Aldan, Ç., Coşkun, E., and Soydan, A. Security testing of web applications: A systematic mapping of the literature. *Journal of King Saud University-Computer and Information Sciences 34*, 9 (2022), 6775–6792.

[8] Betarte, G., Pardo, Á., and Martínez, R. Web application attacks detection using machine learning techniques. In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)* (2018), IEEE, pp. 1065–1072.

[9] Demetrio, L., Biggio, B., Lagorio, G., Roli, F., and Armando, A. Explaining vulnerabilities of deep learning to adversarial malware binaries. *arXiv preprint arXiv:1901.03583* (2019).

[10] Diogenes, Y., and Ozkaya, E. *Cybersecurity-attack and defense strategies: Infrastructure security with red team and blue team tactics*. Packt Publishing Ltd, 2018.

[11] Gandhi, N., Patel, J., Sisodiya, R., Doshi, N., and Mishra, S. A cnn-bilstm based approach for detection of sql injection attacks. In *2021 International conference on computational intelligence and knowledge economy (ICCIKE)* (2021), IEEE, pp. 378–383.

[12] Issakhani, M., Huang, M., Tayebi, M. A., and Lashkari, A. H. An evolutionary algorithm for adversarial sql injection attack generation. In *2023 IEEE International Conference on Intelligence and Security Informatics (ISI)* (2023), pp. 1–6.

[13] Ito, M., and Iyatomi, H. Web application firewall using character-level convolutional neural network. In *2018 IEEE 14th International Colloquium on Signal Processing & Its Applications (CSPA)* (2018), IEEE, pp. 103–106.

[14] Kar, D., Panigrahi, S., and Sundararajan, S. Sqligot: Detecting sql injection attacks using graph of tokens and svm. *Computers & Security 60* (2016), 206–225.

[15] Kim, M.-Y., and Lee, D. H. Data-mining based sql injection attack detection using internal query trees. *Expert Systems with Applications 41*, 11 (2014), 5416–5430.

[16] Kolosnjaji, B., Demontis, A., Biggio, B., Maiorca, D., Giacinto, G., Eckert, C., and Roli, F. Adversarial malware binaries: Evading deep learning for malware detection in executables. In *2018 26th European signal processing conference (EUSIPCO)* (2018), IEEE, pp. 533–537.

[17] Liu, T., Qi, Y., Shi, L., and Yan, J. Locate-then-detect: Real-time web attack detection via attention-based deep neural networks. In *IJCAI* (2019), pp. 4725–4731.

[18] Luo, A., Huang, W., and Fan, W. A cnn-based approach to the detection of sql injection attacks. In *2019 IEEE/ACIS 18th International Conference on Computer and Information Science (ICIS)* (2019), IEEE, pp. 320–324.

[19] Moradi Vartouni, A., Teshnehlab, M., and Sedighian Kashi, S. Leveraging deep neural networks for anomaly-based web application firewall. *IET Information Security 13*, 4 (2019), 352–361.

[20] Prandl, S., Lazarescu, M., and Pham, D.-S. A study of web application firewall solutions. In *Information Systems Security: 11th International Conference, ICISS 2015, Kolkata, India, December 16-20, 2015. Proceedings 11* (2015), Springer, pp. 501–510.

[21] Qu, Z., Ling, X., Wang, T., Chen, X., Ji, S., and Wu, C. Advsqli: Generating adversarial sql injections against real-world waf-as-a-service. *IEEE Transactions on Information Forensics and Security* (2024).

[22] Ramamurthy, R., Ammanabrolu, P., Brantley, K., Hessel, J., Sifa, R., Bauckhage, C., Hajishirzi, H., and Choi, Y. Is reinforcement learning (not) for natural language processing: Benchmarks, baselines, and building blocks for natural language policy optimization. *arXiv preprint arXiv:2210.01241* (2022).

[23] Roberts, A., Raffel, C., Lee, K., Matena, M., Shazeer, N., Liu, P. J., Narang, S., Li, W., and Zhou, Y. Exploring the limits of transfer learning with a unified text-to-text transformer. *Google, Tech. Rep.* (2019).

[24] Rosenberg, I., Shabtai, A., Rokach, L., and Elovici, Y. Generic black-box end-to-end attack against state of the art api call based malware classifiers. In *Research in Attacks, Intrusions, and Defenses: 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings 21* (2018), Springer, pp. 490–510.

[25] Valenza, A., Demetrio, L., Costa, G., and Lagorio, G. Waf-a-mole: An adversarial tool for assessing ml-based wafs. *SoftwareX 11* (2020), 100367.

[26] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems 30* (2017).

[27] Xu, W., Qi, Y., and Evans, D. Automatically evading classifiers: A case study on pdf malware classifiers. ndss, 2016.

[28] Yadav, T., and Rao, A. M. Technical aspects of cyber kill chain. In *Security in Computing and Communications: Third International Symposium, SSCC 2015, Kochi, India, August 10-13, 2015. Proceedings 3* (2015), Springer, pp. 438–452.

[29] Zhang, L., Zhang, D., Wang, C., Zhao, J., and Zhang, Z. Art4sqli: The art of sql injection vulnerability discovery. *IEEE Transactions on Reliability 68*, 4 (2019), 1470–1489.